
ОРГАНІЗАЦІЯ БАЗ ДАНИХ ТА ЗНАНЬ

Групування даних в PostgreSQL

Для групування даних в PostgreSQL застосовуються оператори **GROUP BY** і **HAVING**, для використання яких застосовується наступний формальний синтаксис:

```
1 SELECT столбцы
2 FROM таблица
3 [WHERE условие_фильтрации_строк]
4 [GROUP BY столбцы_для_группировки]
5 [HAVING условие_фильтрации_групп]
6 [ORDER BY столбцы_для_сортировки]
```

Для розгляду операторів візьмемо таку таблицю:

```
1 CREATE TABLE Products
2 (
3     Id SERIAL PRIMARY KEY,
4     ProductName VARCHAR(30) NOT NULL,
5     Company VARCHAR(20) NOT NULL,
6     ProductCount INT DEFAULT 0,
7     Price NUMERIC NOT NULL,
8     IsDiscounted BOOL
9 );
```

GROUP BY

Оператор **GROUP BY** визначає, як рядки будуть групуватися. Наприклад, згрупуємо товари по виробнику:

```
1 SELECT Company, COUNT(*) AS ModelsCount
2 FROM Products
3 GROUP BY Company;
```

	Data Output	Explain	Messages	Query History
	company character varying (20)	modelscount bigint		
1	HTC	1		
2	HMD Global	1		
3	Samsung	2		
4	Apple	3		

Перший стовпець в виразі **SELECT - Company** представляє назву групи, а другий стовпець - **ModelsCount** представляє результат функції **Count**, яка обчислює кількість рядків в групі.

Додамо групування за кількістю товарів:

```
1 SELECT Company, ProductCount, COUNT(*) AS ModelsCount  
2 FROM Products  
3 GROUP BY Company, ProductCount;
```

Слід враховувати, що вираз **GROUP BY** повинен йти після виразу **WHERE**, але до виразу **ORDER BY**:

```
1 SELECT Company, COUNT(*) AS ModelsCount
2 FROM Products
3 WHERE Price > 30000
4 GROUP BY Company
5 ORDER BY ModelsCount DESC;
```

Data Output Explain Messages Query History

	company character varying (20)	modelscount bigint
1	Apple	3
2	Samsung	2
3	HMD Global	1

Фільтрація груп. **HAVING**.

Оператор **HAVING** вказує, які групи будуть включені в вихідний результат, тобто виконує фільтрацію груп. Його використання аналогічно застосуванню оператора **WHERE**.

Наприклад, згрупуємо по виробникам і знайдемо всі групи, для яких визначено більше 1 моделі:

```
1  SELECT Company, COUNT(*) AS ModelsCount
2  FROM Products
3  GROUP BY Company
4  HAVING COUNT(*) > 1;
```

Data Output Explain Messages Query History

	company character varying (20)	modelscount bigint
1	Samsung	2
2	Apple	3

При цьому в одній команді ми можемо використовувати вирази **WHERE** і **HAVING**:

```
1 SELECT Company, COUNT(*) AS ModelsCount
2 FROM Products
3 WHERE Price * ProductCount > 80000
4 GROUP BY Company
5 HAVING COUNT(*) > 1;
```

Якщо при цьому необхідно провести сортування, то вираз **ORDER BY** йде після виразу **HAVING**:

```
1 SELECT Company, COUNT(*) AS Models, SUM(ProductCount) AS Units
2 FROM Products
3 WHERE Price * ProductCount > 80000
4 GROUP BY Company
5 HAVING SUM(ProductCount) > 2
6 ORDER BY Units DESC;
```

Тут групування йде по виробникам, і також вибирається кількість моделей для кожного виробника (Models) і загальна кількість всіх товарів за усіма цими моделями (Units). Потім групи упорядковано відповідно до кількості товарів за спаданням.

1	SELECT Company, COUNT(*) AS Models, ProductCount		
2	FROM Products		
3	GROUP BY GROUPING SETS(Company, ProductCount);		

Data Output Explain Messages Query History			
	company character varying (20)	models bigint	productcount integer
1	HTC	1	[null]
2	HMD Global	1	[null]
3	Samsung	2	[null]
4	Apple	3	[null]
5	[null]	1	3
6	[null]	2	5
7	[null]	1	6
8	[null]	2	2
9	[null]	1	1

Оператор **GROUPING SETS** групує отримані набори окремо.

У виразі **SELECT** проводиться вибірка компаній, кількості моделей і кількості товарів. Тобто ми отримуємо три категорії. Оператор **GROUPING SETS** виробляє угруповання по двох стовпчиках - Company і ProductCount. В результаті буде створюватися дві групи: 1) компанії і кількість моделей і 2) кількість моделей і кількість товарів.

```
1 SELECT Company, COUNT(*) AS Models, ProductCount
2 FROM Products
3 GROUP BY GROUPING SETS (Company, ProductCount);
```

	company character varying (20)	models bigint	productcount integer
1	HTC	1	[null]
2	HMD Global	1	[null]
3	Samsung	2	[null]
4	Apple	3	[null]
5	[null]	1	3
6	[null]	2	5
7	[null]	1	6
8	[null]	2	2
9	[null]	1	1

ROLLUP

Оператор ROLLUP додає підсумовуючий рядок в результуючий набір:

```
1 SELECT Company, COUNT(*) AS Models, SUM(ProductCount) AS Units
2 FROM Products
3 GROUP BY ROLLUP(Company);
```

Data Output Explain Messages Query History

	company character varying (20)	models bigint	units bigint
1	[null]	7	24
2	HTC	1	5
3	HMD Global	1	6
4	Samsung	2	3
5	Apple	3	10

При угрупованню за кількома критеріями **ROLLUP** створюватиме підсумовуючий рядок для кожної з підгруп:

```
1 SELECT Company, COUNT(*) AS Models, SUM(ProductCount) AS Units
2 FROM Products
3 GROUP BY ROLLUP(Company, ProductCount)
4 ORDER BY Company;
```

Data Output Explain Messages Query History

	company character varying (20)	models bigint	units bigint
1	Apple	1	2
2	Apple	1	3
3	Apple	1	5
4	Apple	3	10
5	HMD Global	1	6
6	HMD Global	1	6
7	HTC	1	5
8	HTC	1	5
9	Samsung	1	1
10	Samsung	1	2
11	Samsung	2	3
12	[null]	7	24

CUBE

CUBE схожий на ROLLUP за тим винятком, що CUBE додає підсумовуючі рядки для кожної комбінації груп.

```
1 SELECT Company, COUNT(*) AS Models, SUM(ProductCount) AS Units
2 FROM Products
3 GROUP BY CUBE(Company, ProductCount);
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	company character varying (20)	models bigint	units bigint
1	[null]	7	24
2	Samsung	1	1
3	Samsung	1	2
4	Apple	1	5
5	HTC	1	5
6	Apple	1	3
7	HMD Global	1	6
8	Apple	1	2
9	HTC	1	5
10	HMD Global	1	6
11	Samsung	2	3
12	Apple	3	10
13	[null]	1	3
14	[null]	2	10



